

EC4219: Software Engineering

Lecture 13 — Automatic Type Inference (1) *Generating Type Equations*

Sunbeom So
2025 Spring

The Problem of Automatic Type Inference

Given a program E , infer the most general type of E if E can be typed (i.e., $\emptyset \vdash E : t$ for some $t \in T$). Otherwise, report type errors.

- $\text{let } f = \text{proc } (x) (x + 1) \text{ in } (\text{proc } (x) (x \ 1)) \ f$
- $\text{let } f = \text{proc } (x) (x + 1) \text{ in } (\text{proc } (x) (x \ \text{true})) \ f$
- $\text{proc } (x) \ x$

Automatic Type Inference

- A static analysis algorithm that automatically figures out types of expressions by observing how they are used.
- We will explore the algorithm that is **sound** and **complete** with respect to the type system design.
 - ▶ (Sound) If the analysis finds a type for an expression, the expression is well-typed with the type according to the type system.
 - ▶ (Complete) If an expression has a type according to the type system, the analysis is guaranteed to find the type.
- The algorithm consists of two steps:
 - ① Generate type equations from the program text.
 - ② Solve the equations.

Generating Type Equations

For every subexpression (other than constants), introduce type variables and derive equations between the type variables.

Example 1

$$\begin{array}{c} \text{proc } \underbrace{(f)}_{t_f} \text{ proc } \underbrace{(x)}_{t_x} \underbrace{((f \ 3) - (f \ x))}_{t_2} \\ \underbrace{\hspace{10em}}_{t_1} \\ \underbrace{\hspace{15em}}_{t_0} \end{array}$$

Example 1

$$\underbrace{\text{proc } \underbrace{(f)}_{t_f} \text{ proc } \underbrace{(x)}_{t_x} \underbrace{((f \ 3) - (f \ x))}_{t_2}}_{t_1} \underbrace{\hspace{10em}}_{t_0}$$

$$\begin{aligned} t_0 &= t_f \rightarrow t_1 \\ t_1 &= t_x \rightarrow t_4 \\ t_3 &= \text{int} \\ t_4 &= \text{int} \\ t_2 &= \text{int} \\ t_f &= \text{int} \rightarrow t_3 \\ t_f &= t_x \rightarrow t_4 \end{aligned}$$

cf) These are simplified versions of the equations for brevity.

Example 2

$$\text{proc } \underbrace{\underbrace{(f)}_{t_f} \underbrace{(f \ 11)}_{t_1}}_{t_0}$$

Example 2

$$\underbrace{\text{proc } \underbrace{(f)}_{t_f} \underbrace{(f \ 11)}_{t_1}}_{t_0}$$

$$t_0 = t_f \rightarrow t_1$$

$$t_f = \text{int} \rightarrow t_1$$

Example 3

$$\underbrace{\text{if } \underbrace{x}_{t_x} \text{ then } \underbrace{(x-1)}_{t_1} \text{ else } 0}_{t_0}$$

Example 3

$$\underbrace{\text{if } \underbrace{x}_{t_x} \text{ then } \underbrace{(x-1)}_{t_1} \text{ else } 0}_{t_0}$$

$$t_x = \text{bool}$$

$$t_1 = t_0$$

$$\text{int} = t_0$$

$$t_x = \text{int}$$

$$t_1 = \text{int}$$

cf) we cannot obtain solutions due to the conflict.

Example 4

$\text{proc } \underbrace{(f)}_{t_f} \underbrace{(\text{iszero } \underbrace{(f f)}_{t_2})}_{t_1}$
 $\underbrace{\hspace{10em}}_{t_0}$

Example 4

$$\text{proc } \underbrace{(f)}_{t_f} \underbrace{(\text{iszero } \underbrace{(f f)}_{t_2})}_{t_1}$$

$\underbrace{\hspace{10em}}_{t_0}$

$$t_0 = t_f \rightarrow t_1$$

$$t_1 = \text{bool}$$

$$t_2 = \text{int}$$

$$t_f = t_f \rightarrow t_2$$

cf) we cannot compute solutions due to $t_f = t_f \rightarrow t_2$.

Idea: Deriving Equations based on Typing Rules

The typing rules dictate the equations that must hold between the type variables.

- $$\frac{\Gamma \vdash E_1 : \text{int} \quad \Gamma \vdash E_2 : \text{int}}{\Gamma \vdash E_1 + E_2 : \text{int}}$$

$$t_{E_1} = \text{int} \wedge t_{E_2} = \text{int} \wedge t_{E_1 + E_2} = \text{int}$$

- $$\frac{\Gamma \vdash E : \text{int}}{\Gamma \vdash \text{iszero } E : \text{bool}}$$

$$t_E = \text{int} \wedge t_{(\text{iszero } E)} = \text{bool}$$

- $$\frac{\Gamma \vdash E_1 : t_1 \rightarrow t_2 \quad \Gamma \vdash E_2 : t_1}{\Gamma \vdash E_1 E_2 : t_2}$$

$$t_{E_1} = t_{E_2} \rightarrow t_{(E_1 E_2)}$$

Idea: Deriving Equations based on Typing Rules

$$\bullet \frac{\Gamma \vdash E_1 : \text{bool} \quad \Gamma \vdash E_2 : t \quad \Gamma \vdash E_3 : t}{\Gamma \vdash \text{if } E_1 \text{ then } E_2 \text{ else } E_3 : t}$$

$$\begin{aligned} t_{E_1} &= \text{bool} \wedge \\ t_{E_2} &= t_{(\text{if } E_1 \text{ then } E_2 \text{ else } E_3)} \wedge \\ t_{E_3} &= t_{(\text{if } E_1 \text{ then } E_2 \text{ else } E_3)} \end{aligned}$$

$$\bullet \frac{[x \mapsto t_1] \Gamma \vdash E : t_2}{\Gamma \vdash \text{proc } x \ E : t_1 \rightarrow t_2}$$

$$t_{(\text{proc } (x) \ E)} = t_x \rightarrow t_E$$

$$\bullet \frac{\Gamma \vdash E_1 : t_1 \quad [x \mapsto t_1] \Gamma \vdash E_2 : t_2}{\Gamma \vdash \text{let } x = E_1 \text{ in } E_2 : t_2}$$

$$t_x = t_{E_1} \wedge t_{E_2} = t_{(\text{let } x=E_1 \text{ in } E_2)}$$

So far ...

- We have informally discussed how to derive type equations.
- From now on, we define the procedure precisely.

$$\begin{array}{lcl} E & \rightarrow & n \\ & | & x \\ & | & E + E \\ & | & E - E \\ & | & \text{iszero } E \\ & | & \text{if } E \text{ then } E \text{ else } E \\ & | & \text{let } x = E \text{ in } E \\ & | & \text{proc } x \ E \\ & | & E \ E \\ \\ T & \rightarrow & \text{int} \\ & | & \text{bool} \\ & | & T \rightarrow T \\ & | & \alpha \ (\in \text{TyVar}) \end{array}$$

We extend T with α (type variable) to generate **type equations**.

Type Equations

- Type equations are conjunctions of type-equalities: e.g.,

$$\begin{aligned}t_0 &= t_f \rightarrow t_1 \\t_1 &= t_x \rightarrow t_4 \\t_3 &= \text{int} \\t_4 &= \text{int} \\t_2 &= \text{int} \\t_f &= \text{int} \rightarrow t_3 \\t_f &= t_x \rightarrow t_4\end{aligned}$$

- Type equations ($TyEqn$) are defined inductively:

$$\begin{array}{l}TyEqn \rightarrow \emptyset \\| \quad T \doteq T \wedge TyEqn\end{array}$$

Deriving Type Equations

- Algorithm for generating equations:

$$\mathcal{V} : (Var \rightarrow T) \times E \times T \rightarrow TyEqn$$

- $\mathcal{V}(\Gamma, e, t)$ generates the constraint for e to have type t under Γ :

$$\Gamma \vdash e : t \text{ iff } \mathcal{V}(\Gamma, e, t) \text{ is satisfied.}$$

- Examples:

- $\mathcal{V}([x \mapsto \text{int}], x+1, \alpha) = \alpha \doteq \text{int}$
- $\mathcal{V}(\emptyset, \text{proc } (x) \text{ (if } x \text{ then } 1 \text{ else } 2), \alpha \rightarrow \beta) = \alpha \doteq \text{bool} \wedge \beta \doteq \text{int}$

- To derive type equations for closed expression E , we invoke $\mathcal{V}(\emptyset, E, \alpha)$, where α is a fresh type variable.

Deriving Type Equations

$$\mathcal{V}(\Gamma, n, t) = t \doteq \text{int}$$

$$\mathcal{V}(\Gamma, x, t) = t \doteq \Gamma(x)$$

$$\mathcal{V}(\Gamma, e_1 + e_2, t) = t \doteq \text{int} \wedge \mathcal{V}(\Gamma, e_1, \text{int}) \wedge \mathcal{V}(\Gamma, e_2, \text{int})$$

$$\mathcal{V}(\Gamma, \text{iszero } e, t) = t \doteq \text{bool} \wedge \mathcal{V}(\Gamma, e, \text{int})$$

$$\mathcal{V}(\Gamma, \text{if } e_1 \text{ } e_2 \text{ } e_3, t) = \mathcal{V}(\Gamma, e_1, \text{bool}) \wedge \mathcal{V}(\Gamma, e_2, t) \wedge \mathcal{V}(\Gamma, e_3, t)$$

$$\mathcal{V}(\Gamma, \text{let } x = e_1 \text{ in } e_2, t) = \mathcal{V}(\Gamma, e_1, \alpha) \wedge \mathcal{V}([x \mapsto \alpha]\Gamma, e_2, t) \text{ (new } \alpha)$$

$$\mathcal{V}(\Gamma, \text{proc } (x) \text{ } e, t) = t \doteq \alpha_1 \rightarrow \alpha_2 \wedge \mathcal{V}([x \mapsto \alpha_1]\Gamma, e, \alpha_2) \\ \text{(new } \alpha_1, \alpha_2)$$

$$\mathcal{V}(\Gamma, e_1 \text{ } e_2, t) = \mathcal{V}(\Gamma, e_1, \alpha \rightarrow t) \wedge \mathcal{V}(\Gamma, e_2, \alpha) \text{ (new } \alpha)$$

Example

$$\begin{aligned} & \mathcal{V}(\emptyset, (\text{proc } (x) (x)) \ 1, \alpha) \\ &= \mathcal{V}(\emptyset, \text{proc } (x) (x), \alpha_1 \rightarrow \alpha) \wedge \mathcal{V}(\emptyset, 1, \alpha_1) && \text{new } \alpha_1 \\ &= \alpha_1 \rightarrow \alpha \dot{=} \alpha_2 \rightarrow \alpha_3 \wedge \mathcal{V}([x \mapsto \alpha_2], x, \alpha_3) \wedge \alpha_1 \dot{=} \text{int} && \text{new } \alpha_2, \alpha_3 \\ &= \alpha_1 \rightarrow \alpha \dot{=} \alpha_2 \rightarrow \alpha_3 \wedge \alpha_3 \dot{=} \alpha_2 \wedge \alpha_1 \dot{=} \text{int} \end{aligned}$$

Exercise 1

$$\mathcal{V}(\emptyset, \text{proc}(f)(f \text{ 11}), \alpha)$$

Exercise 2

$$\mathcal{V}([x \mapsto \text{bool}], \text{if } x \text{ then } (x - 1) \text{ else } 0, \alpha)$$

Exercise 3

$$\mathcal{V}(\emptyset, \text{proc } (f) \text{ (iszero } (f \ f)), \alpha)$$

Summary

We have defined the algorithm for deriving type equations from program text:

- Given a program E , call $\mathcal{V}(\emptyset, E, \alpha)$ to derive type equations.
- Solve the equations and find the type assigned to α .